Visitor Pattern

In-memory file system
Java 17 / Maven / JUnit 5

1. Purpose and object model

The application stores a hierarchy of files and directories in memory. Three independent operations use the same tree: total size calculation, file search and statistics collection.

Element defines a name, a child collection and accept(Visitor). **FileSystemNode** stores the name and a private parent reference. **FileNode** stores a long byte size. **DirectoryNode** manages child elements. Files return an empty child collection, so the walker does not need type checks.

2. Double dispatch

First, element.accept(visitor) selects the concrete element implementation at runtime. Inside FileNode, visitor.visit(this) selects the visit(FileNode) overload; inside DirectoryNode, it selects visit(DirectoryNode). The runtime visitor type then determines which operation implementation runs. No type-checking branches, casts or reflection select element behavior.

3. Traversal ownership and state

FileTreeWalker.walk owns the depth-first traversal from the selected root. It calls reset() before each run and setContext() with the current relative path and depth before accept(). Only the walker traverses children. Each reachable node is processed exactly once per operation.

Visitor instances are reusable through walk(): each run clears previous results. The order of independent operations does not affect their answers. A direct accept() call processes one node without resetting state or traversing children. Traversal is sequential; the tree must not change during a traversal.

4. Tree integrity

Constructors reject negative sizes and invalid names. Names must not be blank, contain / or backslash, or equal '.' or '..'. DirectoryNode.add rejects null children, cycles and duplicate sibling names. attachTo rejects a second parent. Cycle detection follows parent references from the receiving directory. Rejected additions leave the tree unchanged. Child collections are returned as immutable snapshots.

Operations and output

5. Visitor operations

TotalSizeVisitor adds file sizes; directories contribute no bytes. Math.addExact reports long overflow instead of returning an incorrect total.

SearchVisitor accepts a Predicate<String> for filenames and an inclusive minimum size. Both conditions must hold. Results are immutable and sorted lexicographically. Paths use / and are relative to the selected root. The root directory name is omitted; a file root uses its filename.

StatisticsVisitor counts files and directories and records the maximum depth of all visited nodes, including empty directories. The root depth is 0, and a directory root contributes to the directory count. The walker increases the depth by one for each parent-child edge.

6. Demonstration data and output

```
workspace/
  readme.txt (120)
  documents/
    notes.txt (80), report.txt (450)
    archive/old.txt (100)
  images/logo.png (2048)
  empty/
```

Search criterion: filenames ending in .txt with size >= 100 bytes.

```
Total size: 2798 bytes
Text files with size >= 100 bytes:
  documents/archive/old.txt
  documents/report.txt
  readme.txt
Files: 5
Directories: 5
Maximum depth: 3
```

7. Five Clean Code practices

Meaningful names: SearchVisitor.minimumSize and StatisticsVisitor.getMaximumDepth describe their constraint and result.

Single Responsibility: FileTreeWalker owns traversal, while SearchVisitor owns file filtering, keeping the responsibilities separate.

Small methods: FileNode.accept contains one delegation call, making the dispatch behavior easy to follow.

Encapsulation: DirectoryNode.getChildren returns List.copyOf, so public tree changes must use the validated add method.

No type-checking chains: FileNode.accept and DirectoryNode.accept call the matching visit(this) overload, selecting behavior through polymorphism.

Verification and extension

8. Automated tests

Executed on Java 17 using mvn test from the project root. **25 tests passed: 0 failures, 0 errors and 0 skipped. BUILD SUCCESS.**

Test class	Tests	Verified behavior
DirectoryNodeTest	7	Sizes, names, duplicates, parents, cycles and immutable collections
FileTreeWalkerTest	2	Single visits, paths, depth, reset and null arguments
StatisticsVisitorTest	5	Counts, roots, empty directories, depth and repeated runs
VisitorDispatchTest	1	Both visit overloads through Element.accept
VisitorOperationsTest	10	Total size, both filters, sorting, subtrees, repeated runs and overflow

T1 is covered by VisitorDispatchTest; T2-T4 by VisitorOperationsTest; T5 by StatisticsVisitorTest; T6-T7 by VisitorOperationsTest and StatisticsVisitorTest; T8 by DirectoryNodeTest. FileTreeWalkerTest also verifies context delivery and exactly one visit per element.

9. Extension boundary and trade-offs

The initial commit contains the elements, Visitor contract, walker and two operations. The next commit adds StatisticsVisitor, its tests and its Main invocation. FileNode, DirectoryNode, Visitor and FileTreeWalker remain unchanged in that extension: the two visit overloads and lifecycle methods already exist in the original contract.

A new operation is implemented as another visitor. Adding a new element type requires a new Visitor.visit overload and updates to every existing visitor. This is the main Visitor trade-off when the set of element types changes.

One walker avoids duplicating traversal logic but requires the reset and setContext lifecycle. Recursion keeps traversal simple, while very deep trees are limited by the Java call stack. Immutable snapshots protect internal state at the cost of copying collections.

10. Commands and repository

```
mvn test
mvn compile
java -cp target/classes kz.aitu.visitor.Main
```

GitHub: anuarmaksut711-dotcom/assignment-6-visitor-pattern

UML Class Diagram: Visitor Pattern

